

PROJECT AND TEAM INFORMATION

Project Title

WebC: An AOT Compiler Transforming HTML and JavaScript into Standalone Native Executables via LLVM Infrastructure

Student / Team Information

<i>Team Name:</i>	KernelSpace
Team member 1 (Team Lead) Name - Aditya Ghildiyal Student Id - 230112716 Email - ghildiyaladitya6@gmail.com Contact - +91 8439906540 University Roll no - 2318207	
Team member 2 Name - Shobhit Taragi Student Id - 230115016 Email - harshutaragi@gmail.com Contact - +91 9149193984 University Roll no - 2319584	
Team member 3 Name - Varun Bahuguna Student Id - 230115040 Email - varunbahuguna2@gmail.com Contact - +91 6398786108 University Roll no - 2319832	
Team member 4 Name - Priyanshu Bahuguna Student Id - 230112687 Email - priyanshubahuguna923@gmail.com Contact - +91 6396119882 University Roll no - 2319300	

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Modern web applications rely on heavy virtual machine execution and runtime interpretation of HTML and JavaScript, creating fundamental limitations in performance, portability, and deployment. This reliance on dynamic translation makes web applications significantly slower than native code for computation-intensive tasks. Furthermore, executing web-based tools currently requires massive browser infrastructure or runtime environments, preventing their use in embedded systems, lightweight CLI environments, or resource-constrained scenarios where a full browser engine is unavailable.

Current solutions face critical gaps. Node.js and Deno execute JavaScript outside browsers but still rely on runtime engines and virtual machine management, requiring full environment installation and producing slower execution than native machine code. Desktop frameworks like Electron package entire browser engines, resulting in 100MB+ executables with massive memory overhead—impractical for lightweight applications. While WebAssembly offers portability, it still operates within a restricted runtime layer rather than as a standalone, OS-integrated binary.

WebC addresses these challenges by implementing a true ahead-of-time (AOT) compiler using the LLVM infrastructure. By treating HTML as a structural blueprint and JavaScript as an imperative source language, WebC lowers these high-level constructs directly into optimized machine code. The result is a standalone executable (<1MB) that operates with native performance and zero dependency on browsers or external runtimes. This effectively bridges the gap between the accessibility of web technologies and the efficiency of systems programming, allowing web-based logic to run as high-performance native CLI tools at the speed of compiled languages like C++.

State of the Art / Current solution (1 pt)

Node.js/Deno: Execute JavaScript via runtime engines using dynamic translation rather than static binary generation. They require pre-installed environments and produce execution speeds lower than native code.

Electron/Tauri: Package web apps with a full browser engine (Chromium). This results in massive binary sizes (50MB–100MB+) and significant memory footprints. The application logic remains managed by a bundled runtime engine rather than running as native machine code.

WebAssembly: Compiles to portable bytecode but still requires a dedicated execution layer or a browser. It does not natively handle HTML structure as a standalone binary and is designed as a compilation target, not a complete native solution.

Transpilers (Babel/TypeScript): Perform source-to-source conversion. They do not produce machine code, and the output remains dependent on external engines for execution.

The Gap: No existing tool compiles the integrated web stack—structure and logic—into a standalone native executable.

WebC's Innovation: The first true web-to-native compiler producing optimized standalone binaries (<1MB) using LLVM. It eliminates virtual machine overhead and provides a high-performance pipeline for translating web languages into native machine instructions.

Project Goals and Milestones (2 pts)

Core Objective: Develop a production-ready compiler that transforms HTML and JavaScript into optimized native executables using LLVM, demonstrating a complete Ahead-of-Time (AOT) compilation pipeline.

Specific Goals:

1. Frontend Development: Implement a Flex lexer and Bison parser to handle HTML/JavaScript syntax, managing multiple language contexts (Island Grammars) within a single source stream.
2. Unified AST: Build Abstract Syntax Trees representing the HTML DOM structure and the JavaScript logic (variables, expressions, control flow).
3. Semantic Analysis: Implement type inference for JavaScript variables, symbol table management across different scopes, and compile-time error detection.
4. IR Generation: Translate AST nodes into LLVM Intermediate Representation, mapping JavaScript operations to native instructions and HTML elements to runtime setup calls.
5. Runtime Library: Create a C++ runtime providing a DOM interface and a Terminal User Interface (TUI) engine for native linking.

Timeline:

- Weeks 1-2: Environment setup and C++ runtime library implementation (DOM structures and ANSI-based TUI rendering logic).
- Weeks 3-4: Flex/Bison frontend development; implementation of the hybrid AST nodes.
- Weeks 5-6: Semantic analysis, symbol table construction, and static type checking.
- Weeks 7-10: LLVM IR generation for variables, arithmetic, control flow, and DOM pointer resolution.
- Weeks 11-12: Optimization passes, final linking pipeline, and performance benchmarking against runtime-based engines.

Project Approach (3 pts)

Technical Strategy: WebC implements a seven-stage Ahead-of-Time (AOT) compilation pipeline following classical compiler design principles.

Stage 1 - Lexical Analysis (Flex): Tokenizes HTML and JavaScript using start-conditions to switch context when encountering script tags.

Stage 2 - Syntax Analysis (Bison): Parses tokens into a hybrid AST. It preserves the hierarchical relationship of the DOM and the logical flow of JavaScript expressions.

Stage 3 - Semantic Analysis: Performs type inference and builds a Global Element Registry. It maps HTML identifiers to memory locations that the compiled logic can access.

Stage 4 - IR Generation: Walks the AST to emit LLVM instructions. It maps variables to stack allocations, arithmetic to native math instructions, and HTML elements to C++ runtime function calls.

Stage 5 - Optimization: Applies LLVM middle-end passes, including memory-to-register promotion, constant folding, and dead code elimination to ensure maximum performance.

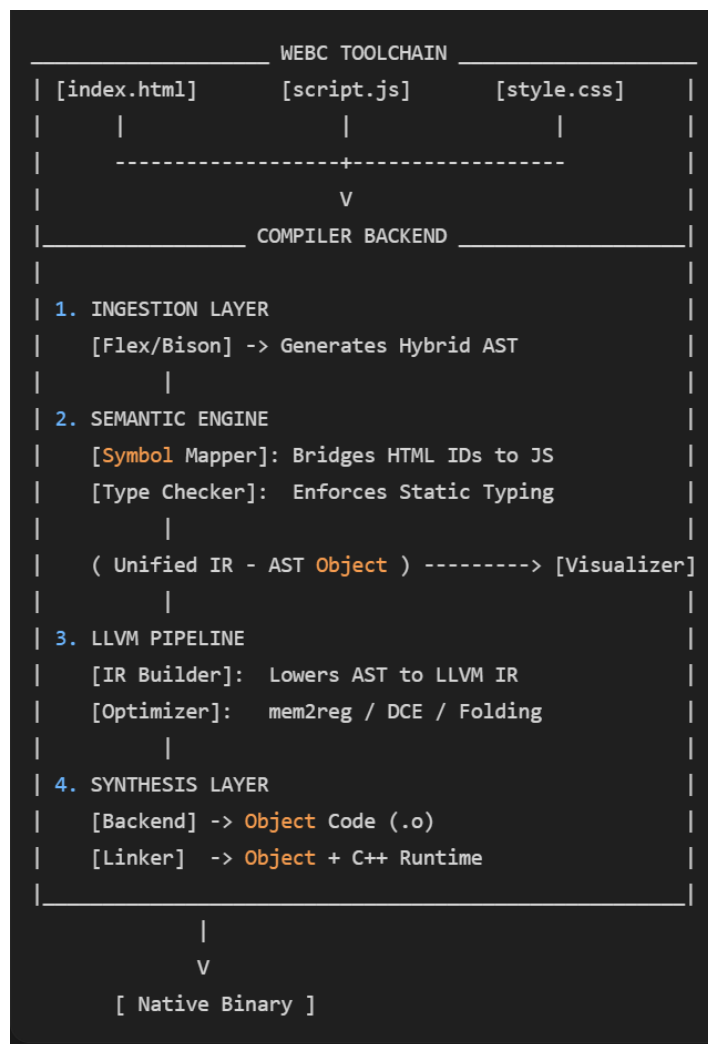
Stage 6 - Code Generation: The LLVM backend generates target-specific machine code (e.g., x86-64) from the optimized IR.

Stage 7 - Linking: The system linker combines the generated object file with the WebC Runtime Library to produce a standalone binary.

TUI Rendering Strategy: To prioritize compiler theory over graphical complexities, the runtime utilizes a Text-based User Interface (TUI). This engine translates HTML/CSS attributes into ANSI escape sequences, rendering the compiled DOM tree directly in the console grid.

Technology Stack: C++17, Flex 2.6+, Bison 3.0+, and LLVM 14+.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

1. **WebC Compiler CLI:** A command-line tool facilitating the full pipeline from source code to native binary, with options for viewing LLVM IR and optimization levels.

2. **C++ Runtime Library:** A lightweight static library providing memory management for the DOM and TUI rendering for the terminal.
3. **Hybrid AST Visualizer:** A tool to export the internal tree structure for debugging and architectural review.
4. **Example Suite:** Working native applications (e.g., Calculator, Counter, Fibonacci Finder) demonstrating native web-tech compilation.
5. **Performance Report:** Documentation comparing the execution speed and memory usage of WebC native binaries against interpretation-based engines.
6. **Technical Documentation:** Grammar specifications (BNF), IR mapping tables, and developer guides.

Assumptions

1. **Language Subset:** Supports core constructs (variables, loops, conditionals) and basic DOM manipulation.
2. **Static Typing:** JavaScript variables follow a strictly-typed model for LLVM compatibility.
3. **Single-File Input:** Focuses on an HTML file with embedded scripts.
4. **Target Architecture:** Initially targets Linux x86-64.
5. **TUI Rendering:** Visual output is managed through a Text-based User Interface (TUI) rather than a graphical windowing system to prioritize core compiler design and terminal-based performance.

References

Compiler Design: - Aho, A. V., et al. "Compilers: Principles, Techniques, and Tools" (Dragon Book) -

<https://www.geeksforgeeks.org/compiler-design-tutorial/>

Academic Papers: - "Source-to-Source Translation and Software Engineering" - IEEE Computer Society - Type Inference Algorithms - Pierce, B. C.

Tools & Libraries: - Abstract Syntax Trees: <https://docs.python.org/3/library/ast.html>

Code Generation Techniques: <https://llvm.org/docs/tutorial/>